

Xdebug Setup & Usage Guide (Laravel + Docker + VS Code)

What is Xdebug?

Xdebug is a PHP debugging tool.

Instead of using:

```
dd($data);
```

everywhere, Xdebug allows you to:

- Pause code execution
- Inspect variables
- Run code line by line
- See the call stack
- Debug APIs, controllers, services, jobs, and middleware

1. Install Xdebug in Docker

Add this to your Dockerfile:

```
# Install Xdebug
RUN pecl install xdebug
    && docker-php-ext-enable xdebug

# Configure Xdebug
RUN echo "xdebug.mode=debug" > /usr/local/etc/php/conf.d/99-xdebug.ini
    && echo "xdebug.start_with_request=yes" >> /usr/local/etc/php/conf.d/99-
xdebug.ini
    && echo "xdebug.client_host=host.docker.internal" >> /usr/local/etc/php/
conf.d/99-xdebug.ini
    && echo "xdebug.client_port=9003" >> /usr/local/etc/php/conf.d/99-xdebug.ini
```

2. Update docker-compose.yml

Inside the PHP container service:

```
extra_hosts:  
  - "host.docker.internal:host-gateway"
```

Example:

```
app:  
  build:  
    context: .  
    dockerfile: Dockerfile  
  
  extra_hosts:  
    - "host.docker.internal:host-gateway"
```

3. Rebuild Docker

```
docker compose down  
  
docker compose build --no-cache  
  
docker compose up -d
```

4. Verify Installation

Check if Xdebug is installed:

```
docker compose exec app php -m | grep xdebug
```

Expected:

```
xdebug
```

Check configuration:

```
docker compose exec app php -i | grep xdebug.mode
```

Expected:

```
xdebug.mode => debug => debug
```

Check:

```
docker compose exec app php -i | grep xdebug.start_with_request
```

Expected:

```
xdebug.start_with_request => yes => yes
```

5. Install VS Code Extension

Install:

PHP Debug (by Xdebug)

Extension ID:

```
xdebug.php-debug
```

6. Create launch.json

Create:

```
.vscode/launch.json
```

Example:

```
{  
  "version": "0.2.0",
```

```
"configurations": [  
  {  
    "name": "Listen for Xdebug",  
    "type": "php",  
    "request": "launch",  
    "port": 9003,  
    "pathMappings": {  
      "/var/www": "${workspaceFolder}/src"  
    }  
  }  
]  
}
```

7. Create Test Route

```
Route::get('/debug-test', function () {  
  
    $name = 'Maruu';  
  
    $age = 25;  
  
    return $name;  
});
```

8. Add Breakpoint

Click left of:

```
$age = 25;
```

You should see a red dot.

This tells Xdebug:

Stop execution here.

9. Start Debugging

Open:

Run & Debug

Select:

Listen for Xdebug

Press:

F5

VS Code starts listening.

10. Trigger the Route

Open:

http://localhost:8000/debug-test

If everything is configured correctly:

```
Browser Request
  ↓
Laravel Route
  ↓
Breakpoint Hit
  ↓
VS Code Stops
```

11. Inspect Variables

When execution pauses:

Hover any variable:

\$name

or

\$age

You can also use:

Variables Panel

on the left side.

12. Useful Shortcuts

Shortcut	Action
F5	Continue
F10	Next Line
F11	Step Into Function
Shift + F11	Step Out
Shift + F5	Stop Debugging

Example

Code:

```
public function test()
{
    $a = 10;

    $b = 20;

    $total = $a + $b;
```

```
    return $total;  
}
```

Breakpoint:

```
$b = 20;
```

Run debugger.

Execution stops before the line executes.

Press:

```
F10
```

to move to the next line.

Now inspect:

```
$total
```

and see its value.

Why Use Xdebug?

Without Xdebug:

```
dd($user);  
dd($posts);  
dd($comments);
```

You keep adding and removing debug statements.

With Xdebug:

- Run once
- Pause anywhere
- Inspect variables
- Continue execution

Much faster and cleaner.